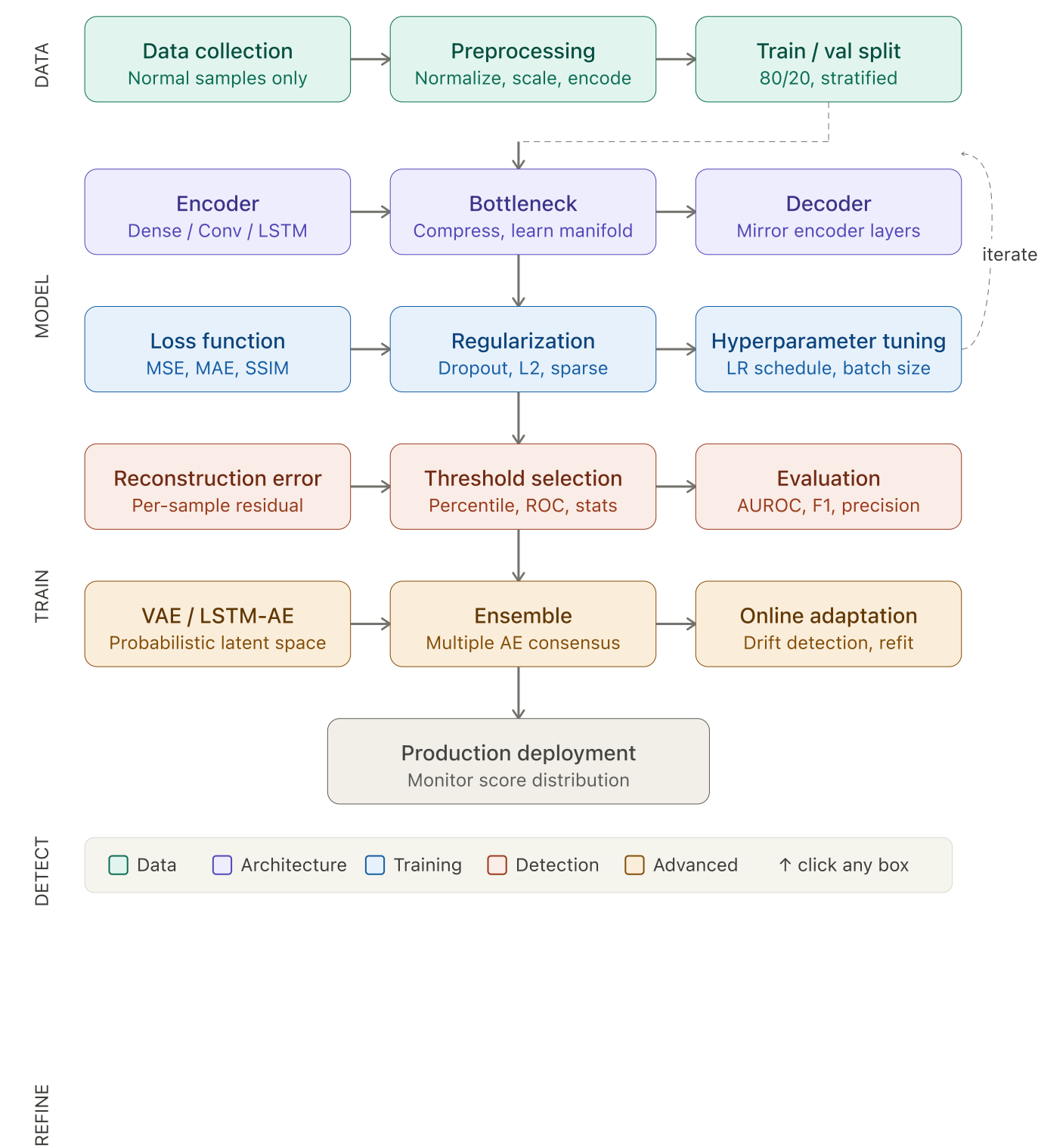


Autoencoders - Anomaly Detection



1. Data strategy

The autoencoder's entire premise is: *train only on normal samples, so anomalies produce high reconstruction error*. If your training data is contaminated with anomalies, the model learns to reconstruct them too, and the signal collapses.

What to do:

- Curate a clean "normal" dataset as your training corpus. Even 1–5% anomaly contamination in training can meaningfully degrade threshold quality.
- Apply **robust normalization**. StandardScaler is fine for Gaussian-ish features; use RobustScaler (median/IQR) when outliers exist in your normal data. Min-Max scaling is dangerous here because anomalies in your test set may exceed the training range.
- For tabular data: encode categoricals (ordinal or embeddings), handle missing values before scaling, and consider feature selection to drop noise dimensions.
- For time series: use sliding window sequences as inputs, not raw timesteps. Window size is a key hyperparameter, too short and you miss temporal patterns, too long and reconstruction becomes easy for everything.
- Split: 80% train, 20% validation, all normal. Keep a completely separate *labeled* test set with real anomaly examples for final evaluation only.

2. Architecture design

The right architecture depends entirely on our data modality:

Data type	Recommended architecture
Tabular / flat features	Deep dense autoencoder (3–5 layers)
Images	Convolutional AE (strided convs + transposed convs)

Data type	Recommended architecture
Time series / sequences	LSTM-AE or 1D Conv-AE
Graph data	Graph AE (GCN encoder)

Bottleneck sizing is the single most critical architectural decision. Too large → the model memorizes everything, anomalies are reconstructed well too. Too small → underfits even normal samples. A useful heuristic: start at 10–30% of input dimensionality, then tune empirically by watching validation reconstruction loss. The bottleneck should be *just big enough* to reconstruct normals faithfully.

Symmetric depth: Mirror the encoder in the decoder (same number of layers, reversed widths). Asymmetric decoders sometimes work better for very structured data, but symmetric is a safe default.

Activation functions: ReLU or Leaky ReLU in hidden layers. Output activation depends on input scaling, sigmoid for [0,1] inputs, linear (no activation) for standardized inputs.

3. Loss function selection

This is where most tutorials oversimplify. MSE is the default but not always right:

- **MSE:** penalizes large errors heavily (squares them), making it sensitive to big deviations. Great when anomalies manifest as large magnitude changes. Weakness: sensitive to noise in normal data.
- **MAE:** more robust to small normal-data noise, but less sensitive to large anomaly spikes.
- **SSIM-based loss** (for images): captures structural/perceptual differences, outperforms pixel-wise MSE for image anomaly detection significantly.
- **KL divergence component** (for VAEs): forces the latent space to be smooth/regular, which makes anomaly latent codes more distinctly out-of-distribution.
- **Hybrid: $0.5 \times \text{MSE} + 0.5 \times \text{MAE}$:** often the best practical choice for tabular data; balances sensitivity and robustness.

For very high-dimensional inputs (images), consider a **perceptual loss** using a pre-trained feature extractor (VGG or ResNet features), reconstruction error in feature space often separates anomalies better than pixel space.

4. Training practices

Learning rate: Use a warmup schedule + cosine decay. Start with $1e-4$ as a baseline. Learning rate is the most impactful single hyperparameter, too high and the model gets stuck in bad minima, too low and it underfits.

Batch size: Larger batches (256–1024) give more stable gradient estimates and tend to produce sharper reconstruction boundaries. Avoid tiny batches (<32) for tabular AEs.

Early stopping: Monitor validation reconstruction loss with patience of 10–20 epochs. The model should *not* be trained to convergence, slightly under-fitted models generalize the normal manifold better than perfectly-converged ones.

Regularization:

- Dropout (rate 0.1–0.2) in encoder layers only. Don't apply dropout in the bottleneck or decoder for standard AEs.
- L2 weight decay ($1e-5$ to $1e-4$) prevents weight explosion on low-variance features.
- **Sparse autoencoders** (add L1 penalty on bottleneck activations) force the model to use fewer latent dimensions per sample, which can sharpen the normal manifold boundary.

Data augmentation for normals (often overlooked): Augmenting your normal training data (gaussian noise injection, small random perturbations, mixup) improves robustness and prevents the encoder from overfitting to exact training samples. This is especially important when normal training data is limited.

5. Threshold calibration - where accuracy is actually won or lost

This step is equally important as the model itself. A perfect model with a bad threshold will have poor detection performance.

How reconstruction error becomes a score: For each sample, compute per-feature squared error, then either sum or take the mean across features. For sequences, average across timesteps too.

Threshold strategies, ranked by quality:

1. **ROC curve + F1 maximization:** if you have any labeled anomaly examples (even a small held-out set), sweep the threshold across all percentiles, compute precision/recall/F1 at each, and pick the threshold that maximizes F1. This directly optimizes the metric you care about.
2. **Statistical threshold on validation normals:** compute mean (μ) and std (σ) of reconstruction errors on validation normal data. Set threshold at $\mu + k \times \sigma$. $k=2.5$ corresponds to ~99th percentile; $k=3$ is ~99.7th. Tune k using domain knowledge about acceptable false positive rate.

- 3. **Percentile threshold:** set threshold at the 95th or 99th percentile of validation reconstruction errors. Simple but effective when you have no labeled anomalies.
- 4. **Peak-over-threshold (Extreme Value Theory):** fit a Generalized Pareto Distribution to the tail of reconstruction errors. Principled for choosing thresholds when anomalies are rare and you want statistical guarantees.

Feature-wise thresholds: Instead of a global threshold on total reconstruction error, set per-feature thresholds. This localizes anomalies (tells you *which* feature is anomalous) and can substantially improve precision, at the cost of more calibration work.

6. Evaluation metrics

Never use accuracy for anomaly detection. Class imbalance makes it meaningless.

- **AUROC:** primary metric. Measures discriminative ability across all thresholds. Target >0.90 for strong performance, >0.95 for excellent.
- **AUPRC (area under precision-recall curve):** more informative than AUROC under extreme class imbalance. Should be your second primary metric.
- **F1 at operating threshold:** practical metric for deployment. Tune threshold to maximize this.
- **False positive rate:** in production, FPR often matters more than F1. An anomaly detector alerting 100× per day becomes noise.

7. Production considerations

Monitoring score drift: The reconstruction error distribution will shift as your production data distribution evolves. Track the rolling mean and percentiles of your anomaly scores. If the p99 score is trending up, your normal distribution has drifted, trigger retraining.

Online adaptation: Periodically retrain (or fine-tune) on recent confirmed-normal samples. Use a sliding window of recent data for rolling retraining. Important: always validate that the new model's AUROC on your held-out test set hasn't regressed before swapping it in.

Latency: For real-time systems, the encoder forward pass is the bottleneck. Quantize or distill the model if inference latency is constrained. Dense AEs on tabular data are typically <1ms even at full precision; Conv-AEs on images may need TensorRT optimization.

Appendix

Window Size

The autoencoder doesn't look at one reading and decide if it's anomalous. It looks at a **chunk of consecutive readings** and asks: *"does this shape look normal?"*

```
WINDOW_SIZE = 30
```

A window of size 30 means: take 30 consecutive readings at a time.

```
readings: [33, 34, 35, 33, 32, 34, 35, 36, 33, 34, ... (30 values)]
           ↓
           this entire chunk = one training sample
```

Then slide forward by 1 and take the next chunk:

```
readings: [34, 35, 33, 32, 34, 35, 36, 33, 34, 35, ... (30 values)]
           ↓
           this entire chunk = next training sample
```

Why not just feed one value at a time?

A single value of CPU=45% tells you nothing — that's normal. But if you see CPU jump from 35% → 99% and *stay there for 30 readings*, that pattern is what screams anomaly.

The model learns the **shape and rhythm** of normal behaviour over time, not just individual values.